



Proyecto final: Infraestructura en AWS
Cloud Computing

Daniel Alejandro Yepes Mesa

Docente
Juan Pablo Arango

Ingeniería de sistemas
Facultad de ingeniería
Universidad de Antioquia
2026

1. Descripción de la solución

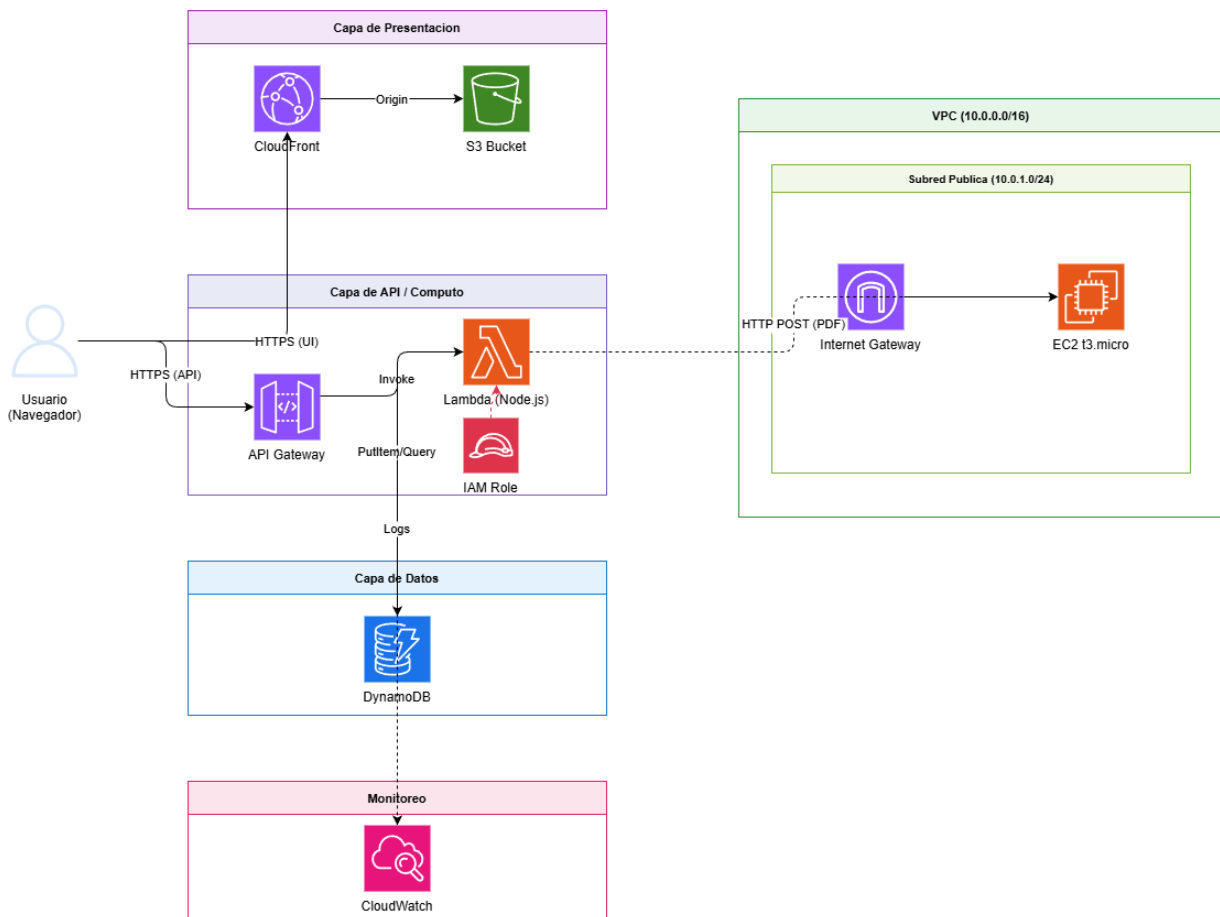
Quotick es una aplicación web para cotizar proyectos de software por horas. La idea nació como una necesidad real: construir propuestas comerciales de forma rápida, consistente y con los números correctos desde la primera vez. El proyecto sirve como caso de estudio para aplicar los conceptos del curso: infraestructura en la nube, computación serverless, almacenamiento, redes y seguridad.

La aplicación permite registrar clientes, definir un tarifario de roles, cargar herramientas y factores de riesgo, y generar cotizaciones con precios calculados automáticamente en COP. Al final de cada cotización se puede descargar un PDF con el resumen comercial listo para enviar al cliente.

Todo el sistema corre en AWS bajo una arquitectura híbrida: serverless para el API y la base de datos, y una instancia EC2 para el único componente que lo requiere técnicamente (la generación de PDF con Puppeteer).

2. Arquitectura y diagrama

Arquitectura Quotick — AWS



La arquitectura tiene tres capas bien separadas: el frontend estático servido desde S3 + CloudFront, el backend compuesto por funciones Lambda expuestas a través de API Gateway, y la base de datos en DynamoDB. Adicionalmente, una instancia EC2 t3.micro actúa como servidor de generación de PDF.

2.1 Servicios utilizados

Servicio AWS	Rol en la arquitectura
S3 (quotick-frontend-prod)	Aloja el frontend compilado (HTML, JS, CSS)
CloudFront	CDN: distribuye el frontend y termina TLS
API Gateway HTTP API	Expone los 25 endpoints REST del backend
Lambda (Node.js 22.x)	Lógica de negocio: 25 funciones, una por endpoint
DynamoDB On-demand	Almacena clientes, servicios, herramientas, factores y cotizaciones
EC2 t3.micro	Servidor HTTP con Puppeteer para generar PDFs
S3 (pdfs/)	Guarda los PDFs generados; Lambda devuelve URL pre-firmada (1 h)
VPC + Subred pública	Red privada donde vive la instancia EC2
IAM	Roles con mínimo privilegio para Lambda y EC2
CloudWatch	Logs y métricas de todas las funciones Lambda
Elastic IP	IP fija para que Lambda siempre llegue a la EC2

2.2 Flujo de datos principal

Flujo general:

1. El usuario abre la app en el navegador
2. React llama al API Gateway
3. API Gateway invoca la Lambda correspondiente
4. Lambda lee o escribe en DynamoDB
5. Lambda devuelve la respuesta y React la muestra

Flujo PDF:

1. El usuario hace clic en "Descargar PDF"
2. Lambda lee la cotización de DynamoDB y la envía a la EC2 vía HTTP POST
3. EC2 renderiza el HTML con Puppeteer, sube el PDF a S3 y genera una URL pre-firmada
4. Lambda devuelve esa URL a React y el navegador abre el PDF

3. Infraestructura: red y seguridad

3.1 VPC y red

Cree una VPC con CIDR 10.0.0.0/16 y una subred pública 10.0.1.0/24 en us-east-1. Configuré un Internet Gateway asociado a la VPC y una Route Table que dirige el tráfico 0.0.0.0/0 hacia ese gateway. La instancia EC2 vive en esta subred con una Elastic IP fija (54.236.139.181) para que la URL de conexión no cambie entre reinicios.

The screenshot displays the AWS Management Console interface for a VPC and its associated Internet Gateway. The top navigation bar shows the AWS logo, a search bar, and the current region (Estados Unidos (Norte de Virginia)). The left sidebar contains the navigation menu with categories like 'Nube virtual privada' and 'Seguridad'. The main content area is divided into two sections: 'subnet-Offeaa243eef284f1 / quotick-public-subnet' and 'igw-059fc10bf24088a38 / quotick-igw'.

subnet-Offeaa243eef284f1 / quotick-public-subnet

Panel de VPC

- AWS Global View
- Filtrar por VPC
- Nube virtual privada
 - Sus VPC
 - Subredes
 - Tablas de enrutamiento
 - Puertas de enlace de Internet
 - Puerta de enlace de Internet de solo salida
 - Gateways de operador
 - Conjuntos de opciones de DHCP
 - Direcciones IP elásticas
 - Listas de prefijos administradas
 - Gateways NAT
 - Interconexiones
 - Servidores de ruta
- Seguridad
 - ACL de red

Detalles

Detalle	Valor
ID de subred	subnet-Offeaa243eef284f1
CIDR IPv4	10.0.1.0/24
Zona de disponibilidad	us-east-1a
ARN de subred	arn:aws:ec2:us-east-1:380607194543:subnet/subnet-Offeaa243eef284f1
Direcciones IPv4 disponibles	250
Grupo de borde de red	us-east-1
Subred predeterminada	No
Asignación automática de direcciones IPv4 propiedad del cliente	No
Grupo IPv4 propiedad del cliente	No
Solo IPv6	No
Reservas de CIDR 6 IPv	No
Registro DNS AAAA del nombre del recurso	Desactivado
ARN de subred	arn:aws:ec2:us-east-1:380607194543:subnet/subnet-Offeaa243eef284f1
Direcciones IPv4 disponibles	250
Grupo de borde de red	us-east-1
Subred predeterminada	No
Asignación automática de direcciones IPv4 propiedad del cliente	No
Grupo IPv4 propiedad del cliente	No
Solo IPv6	No
Reservas de CIDR 6 IPv	No
Registro DNS AAAA del nombre del recurso	Desactivado
Estado	Available
CIDR IPv6	-
VPC	vpc-064e241e35ce3db72 quotick-vpc
Asignación automática de la dirección IPv4 pública	Sí
ID de Outpost	-
Tipo de nombre de alojamiento	Nombre de IP
Propietario	380607194543
Bloquear el acceso público	Desactivado
ID de asociación de CIDR IPv6	-
Tabla de enrutamiento	rtb-09739709e0b22717c quotick-rt-public
Asignar dirección IPv6 automáticamente	No
Reservas de CIDR 4 IPv	-
Registro DNS A del nombre del recurso	Desactivado

Registros de flujo | **Tabla de enrutamiento** | **ACL de red** | **Reservas de CIDR** | **Uso compartido** | **Etiquetas**

Registros de flujo

igw-059fc10bf24088a38 / quotick-igw

Panel de VPC

- AWS Global View
- Filtrar por VPC
- Nube virtual privada
 - Sus VPC
 - Subredes
 - Tablas de enrutamiento
 - Puertas de enlace de Internet
 - Puerta de enlace de Internet de solo salida
 - Gateways de operador
 - Conjuntos de opciones de DHCP
 - Direcciones IP elásticas
 - Listas de prefijos administradas
 - Gateways NAT
 - Interconexiones
 - Servidores de ruta
- Seguridad
 - ACL de red

Detalles

Detalle	Valor
ID de gateway de Internet	igw-059fc10bf24088a38
Estado	Attached
ID de la VPC	vpc-064e241e35ce3db72 quotick-vpc
Propietario	380607194543

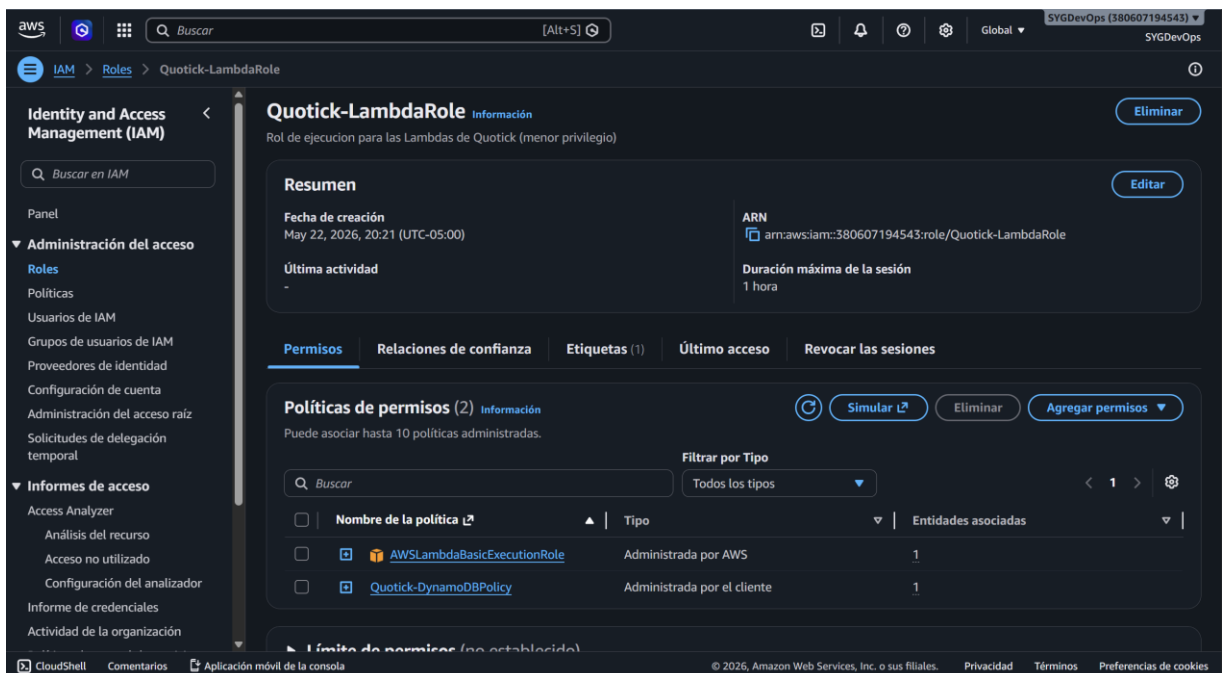
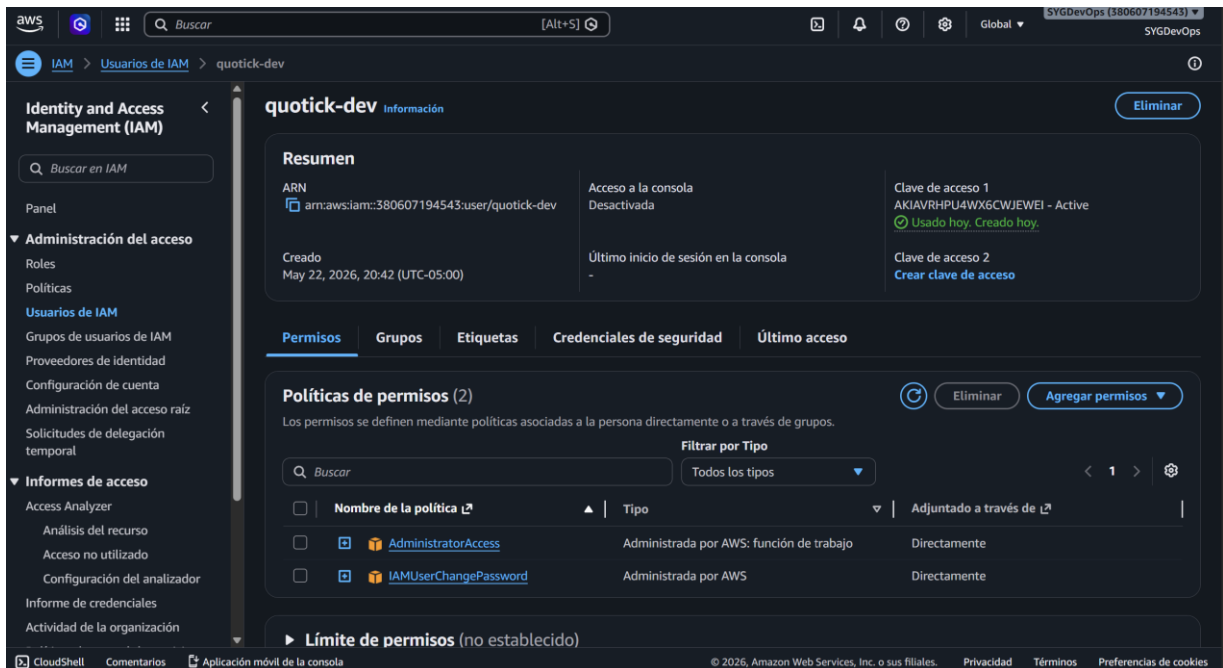
Etiquetas (1)

Clave	Valor
Name	quotick-igw

3.2 IAM — menor privilegio

Cree dos roles IAM específicos para este proyecto. Ninguno tiene permisos de administrador; cada uno solo puede hacer lo que necesita.

Rol	Permisos	Usado por
Quotick-LambdaRole	dynamodb:* sobre tablas quotick-* cloudwatch:PutMetricData logs:CreateLogGroup / PutLogEvents	Todas las funciones Lambda
Quotick-EC2Role	s3:PutObject sobre quotick-frontend-prod/pdfs/* ssm:* (SSM Session Manager)	Instancia EC2

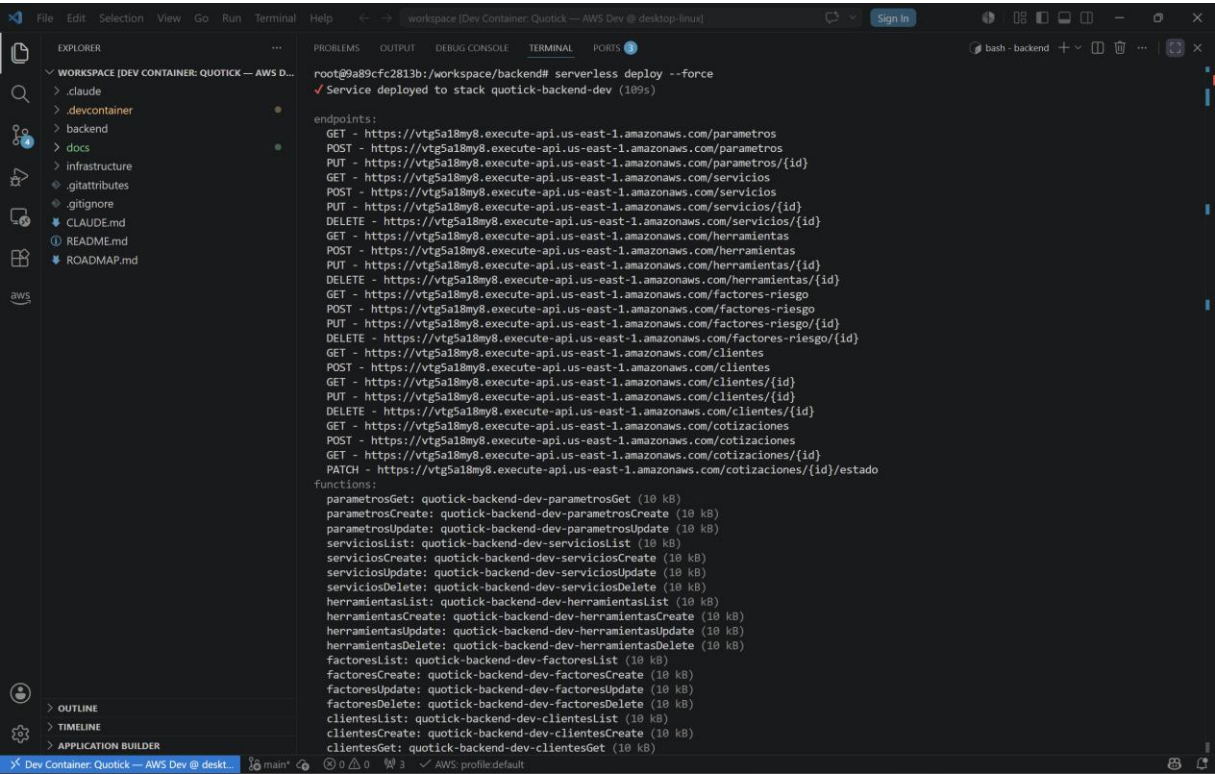


4. Backend serverless

4.1 Lambda + API Gateway

Desarrollé 24 funciones Lambda en Node.js, organizadas por entidad, cada función tiene su propio handler y se expone como un endpoint REST a través de API Gateway HTTP API.

Usé Serverless Framework v3 como herramienta de IaC: un archivo serverless.yml describe todas las funciones, rutas y variables de entorno. Desplegar el backend completo es un solo comando: serverless deploy.



```
root@9a89cfc2813b:/workspace/backend# serverless deploy --force
✔ Service deployed to stack quotick-backend-dev (109s)

endpoints:
GET - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/parametros
POST - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/parametros
PUT - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/parametros/{id}
GET - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/servicios
POST - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/servicios
PUT - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/servicios/{id}
DELETE - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/servicios/{id}
GET - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/herramientas
POST - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/herramientas
PUT - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/herramientas/{id}
DELETE - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/herramientas/{id}
GET - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/factores-riesgo
POST - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/factores-riesgo
PUT - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/factores-riesgo/{id}
DELETE - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/factores-riesgo/{id}
GET - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/clientes
POST - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/clientes
GET - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/clientes/{id}
PUT - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/clientes/{id}
DELETE - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/clientes/{id}
GET - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/cotizaciones
POST - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/cotizaciones
GET - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/cotizaciones/{id}
PATCH - https://vtg5a18my8.execute-api.us-east-1.amazonaws.com/cotizaciones/{id}/estado

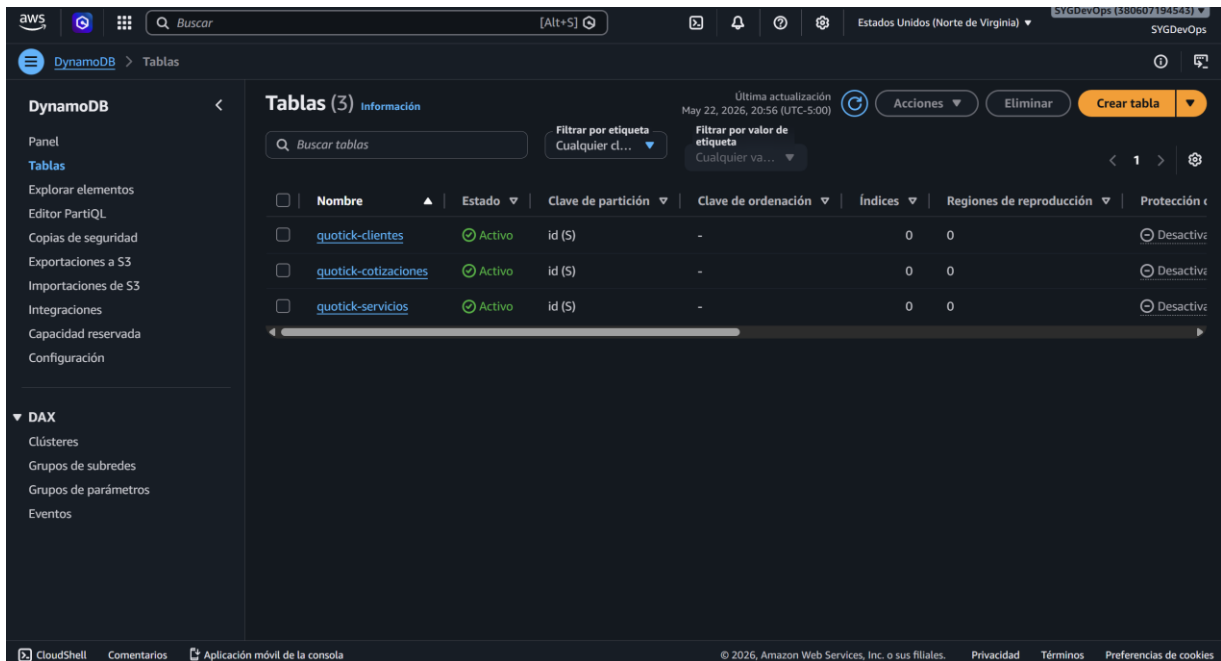
functions:
parametrosGet: quotick-backend-dev-parametrosGet (10 kB)
parametrosCreate: quotick-backend-dev-parametrosCreate (10 kB)
parametrosUpdate: quotick-backend-dev-parametrosUpdate (10 kB)
serviciosList: quotick-backend-dev-serviciosList (10 kB)
serviciosCreate: quotick-backend-dev-serviciosCreate (10 kB)
serviciosUpdate: quotick-backend-dev-serviciosUpdate (10 kB)
serviciosDelete: quotick-backend-dev-serviciosDelete (10 kB)
herramientasList: quotick-backend-dev-herramientasList (10 kB)
herramientasCreate: quotick-backend-dev-herramientasCreate (10 kB)
herramientasUpdate: quotick-backend-dev-herramientasUpdate (10 kB)
herramientasDelete: quotick-backend-dev-herramientasDelete (10 kB)
factoresList: quotick-backend-dev-factoresList (10 kB)
factoresCreate: quotick-backend-dev-factoresCreate (10 kB)
factoresUpdate: quotick-backend-dev-factoresUpdate (10 kB)
factoresDelete: quotick-backend-dev-factoresDelete (10 kB)
clientesList: quotick-backend-dev-clientesList (10 kB)
clientesCreate: quotick-backend-dev-clientesCreate (10 kB)
clientesGet: quotick-backend-dev-clientesGet (10 kB)
```

4.2 DynamoDB

Cree tres tablas DynamoDB en modo On-demand, para un escalamiento automático y sin costos cuando no tiene uso.

Tabla	Partition Key	Contenido
quotick-clientes	id (String)	Datos de clientes registrados
quotick-servicios	id (String)	Tarifario de roles con tarifa por hora en USD
quotick-cotizaciones	id (String)	Cabecera + ítems + precio calculado en COP

Además de eso, cree tablas auxiliares para herramientas, factores de riesgo y parámetros globales, todos estos valores son editables desde la aplicación, sin tocar código.



4.3 Flujo completo

El flujo más importante es la creación de una cotización: el usuario completa el formulario en React, selecciona roles con horas estimadas y elige herramientas y factores de riesgo. React hace un POST a /cotizaciones, API Gateway invoca la Lambda de creación, que lee los parámetros actuales de DynamoDB (TRM, horas/día, GM objetivo), calcula el precio fijo en COP y guarda la cotización completa. La respuesta llega al navegador en menos de un segundo.

← → ↻ dymawbi4thpo.cloudfront.net/nueva

Quotick
Cotizador de Software

PRINCIPAL
Dashboard
Nueva Cotización
Cotizaciones
GESTION
Clientes
Configuración

Nueva Cotización

Datos del proyecto

Cliente

Juan Perez - TechCo SAS

Nombre del proyecto

Portal web Viajes

Escenario de tarifas

Tarifas actuales

Infraestructura mensual (COP)

800000

Roles del equipo

+ Agregar rol

ROL	HEADCOUNT	HORAS TOTALES	
Backend	1	78	✕
Frontend	1	52	✕

Herramientas / Licencias

☒ Jira Service Management (Standard)
USD 23.8/seat × 1 seat

☒ Confluence (Standard)
USD 5.42/seat × 1 seat

☒ Bitbucket (Standard)
USD 3.65/seat × 1 seat

☐ Trello Premium

← → ↻ dymawbi4thpo.cloudfront.net/nueva

Quotick
Cotizador de Software

PRINCIPAL
Dashboard
Nueva Cotización
Cotizaciones
GESTION
Clientes
Configuración

☐ SLA alto / guardias / penalidades (1 pt)
Disponibilidad, tiempos de respuesta.

☒ Seguridad avanzada / auditoria (1 pt)
Permisos complejos, auditoria, compliance.

☐ Legacy / deuda técnica significativa (1 pt)
Código viejo, refactors grandes, falta de tests.

☐ Scraping / anti-bot / fuentes inestables (2 pts)
Cambios de HTML, bloqueos, captchas.

Calcular y Guardar Cotización

Cotización #F3F90D7B

Duración	Puntaje de riesgo	Margen (GM)
0.51 meses	3 pts	50%
Costo mano de obra	Costo herramientas	Costo infraestructura
\$ 4.062.500	\$ 100.227	\$ 408.000
Costo total	Utilidad	Estado
\$ 4.570.727	\$ 4.570.727	borrador

PRECIO FIJO DE VENTA

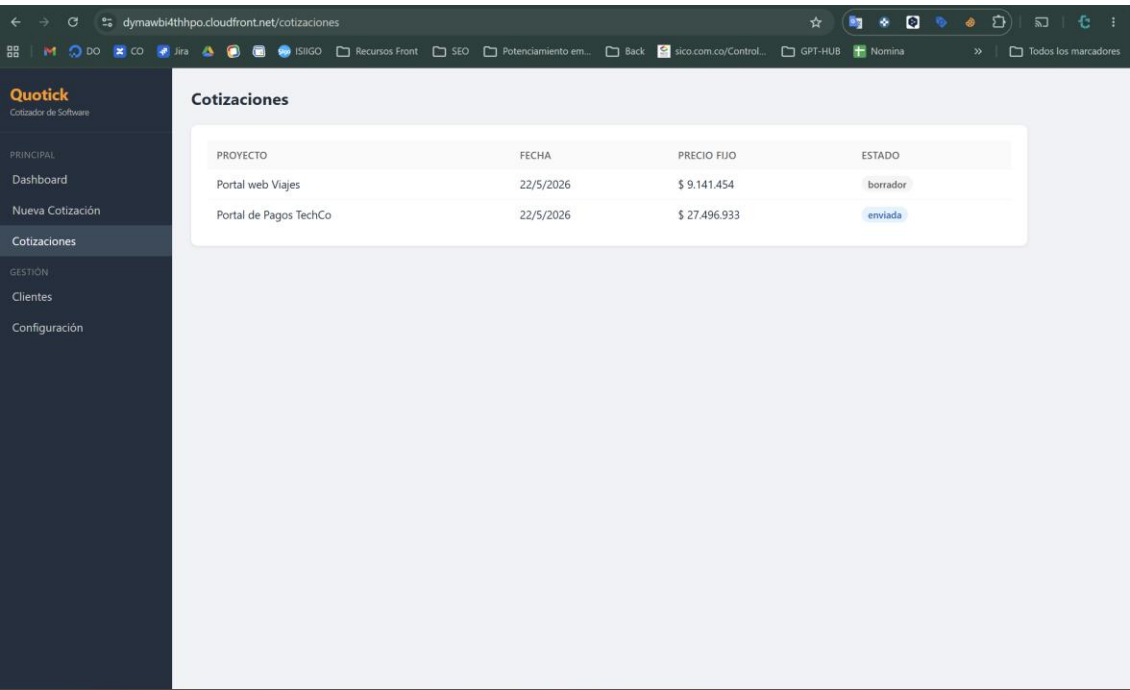
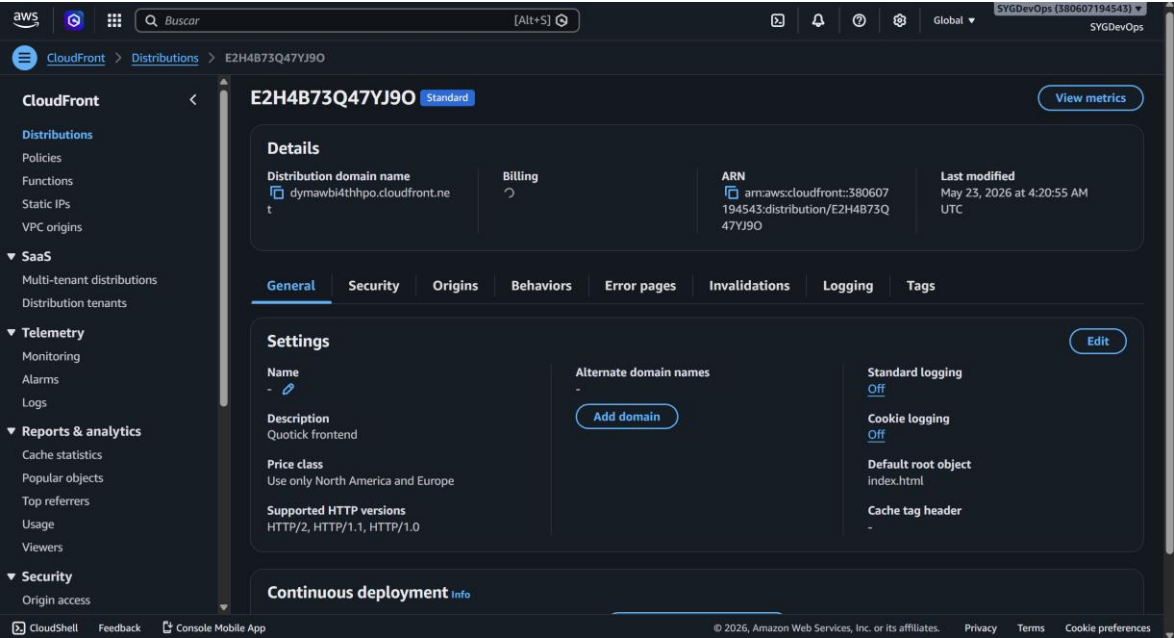
\$ 9.141.454

Precio / mes	Precio / día	Precio / hora
\$ 17.924.420	\$ 896.221	\$ 112.028

5. Frontend

Construí el frontend con React y lo compilé a archivos estáticos (index.html, bundle JS, CSS). Subí el build a un bucket S3 con hosting web habilitado y configuré una

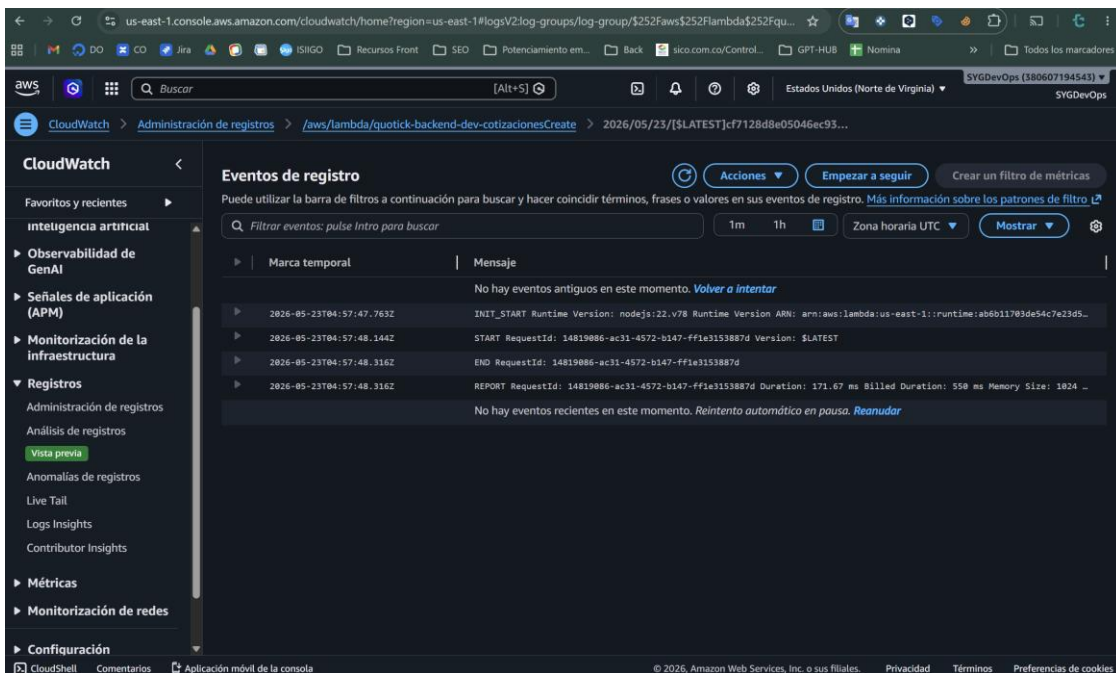
distribución de CloudFront apuntando a ese bucket como origen. CloudFront actúa como CDN: sirve los archivos desde el Edge más cercano al usuario, termina TLS y reduce la latencia. La variable de entorno VITE_API_URL apunta al endpoint de API Gateway, así el frontend sabe a dónde mandar las peticiones.



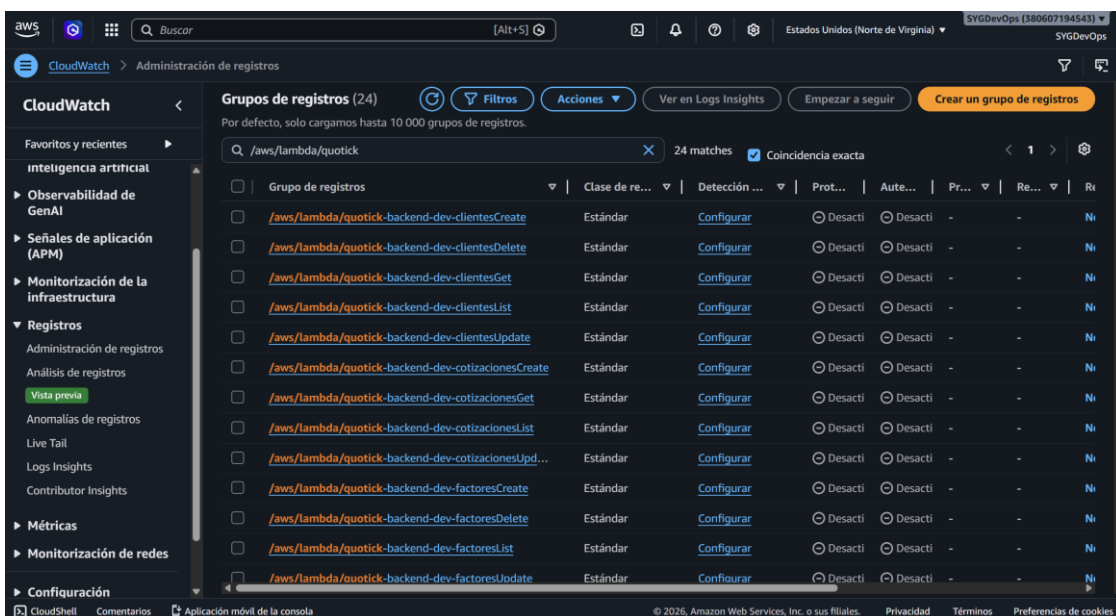
6. Monitoreo

Lambda envía logs automáticamente a CloudWatch Logs. Cada función tiene su propio grupo de logs con el patrón `/aws/lambda/quotick-backend-prod-{función}`. Desde ahí puedo ver cada invocación: duración, memoria usada, errores y el output que las funciones imprimen al ejecutarse.

Configuré métricas básicas de CloudWatch para las funciones más críticas como invocaciones totales, errores y duración promedio. Esto permite detectar si algún endpoint empieza a fallar o a responder lento sin tener que revisar los logs manualmente.



The screenshot shows the AWS CloudWatch console. The left sidebar contains navigation options like 'Favoritos y recientes', 'Inteligencia artificial', 'Observabilidad de GenAI', 'Señales de aplicación (APM)', 'Monitorización de la infraestructura', 'Registros', 'Métricas', 'Monitorización de redes', and 'Configuración'. The main area is titled 'Eventos de registro' (Log Events). It shows a search bar with the filter '2026/05/23[\$LATEST]cf7128d8e05046ec93...'. Below the search bar, there's a table with columns 'Marca temporal' (Timestamp) and 'Mensaje' (Message). The table displays several log events, including 'INIT_START', 'START', 'END', and 'REPORT' messages, each with associated metadata like 'RequestId', 'Duration', and 'Memory Size'.

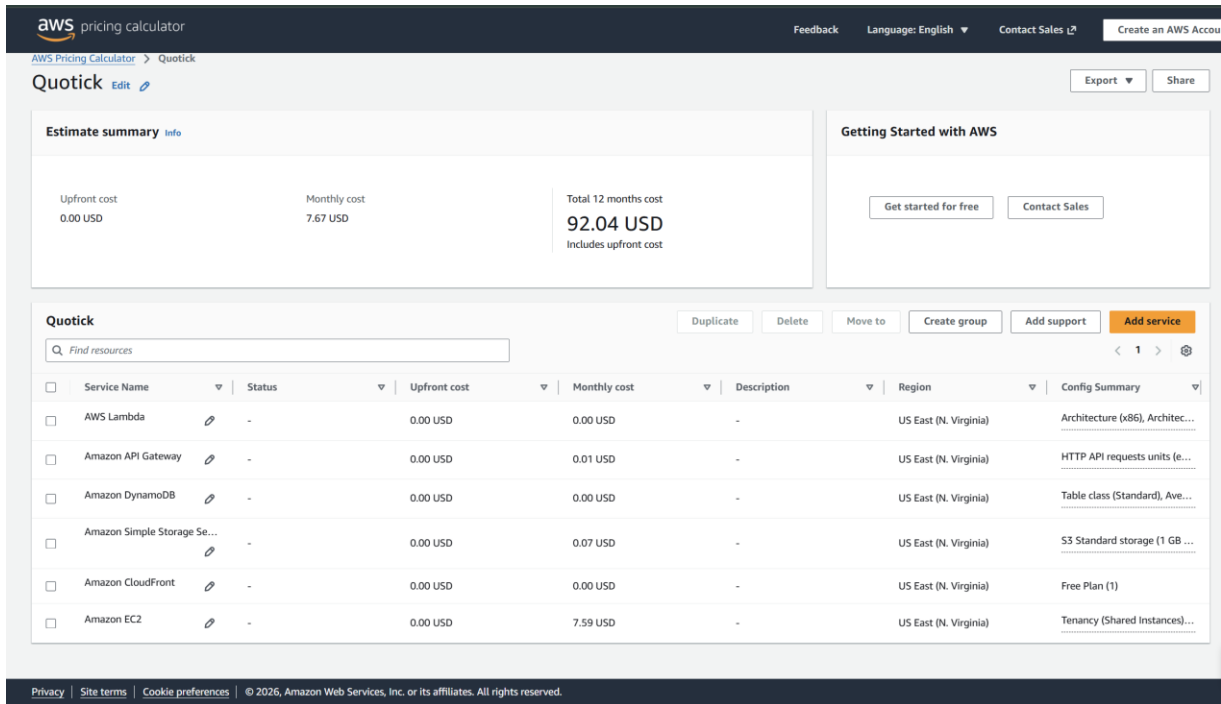


The screenshot shows the AWS CloudWatch console with the 'Grupos de registros' (Log Groups) view. The left sidebar is the same as in the previous screenshot. The main area is titled 'Grupos de registros (24)' (Log Groups (24)). It shows a search bar with the filter '/aws/lambda/quotick'. Below the search bar, there's a table with columns: 'Grupo de registros' (Log Group), 'Clase de re...' (Class of re...), 'Detección ...' (Detection ...), 'Prot...' (Protection), 'Aute...' (Authentication), 'Pr...' (Permissions), 'Re...' (Retention), and 'Ri...' (Resources). The table lists 24 log groups, each with a 'Configurar' (Configure) link. The log groups are named with the pattern '/aws/lambda/quotick-backend-dev-...' followed by the function name.

8. Costos estimados

El costo mensual estimado del proyecto es de **\$7.67 USD/mes**. Lambda, DynamoDB y CloudFront están en \$0.00 dentro del free tier. API Gateway suma \$0.01 y S3 \$0.07 por el almacenamiento del frontend y los PDFs. El único costo real es la instancia EC2 t3.micro corriendo 24/7 con \$7.59/mes, que es lo que justifica tener el generador de PDF siempre disponible.

El único costo fijo real es la instancia EC2 y su IP elástica. Los componentes serverless prácticamente no cuestan nada al volumen de uso actual



The screenshot displays the AWS Pricing Calculator interface for a project named 'Quotick'. The 'Estimate summary' section shows an upfront cost of \$0.00 USD, a monthly cost of \$7.67 USD, and a total 12-month cost of \$92.04 USD. The 'Getting Started with AWS' section includes buttons for 'Get started for free' and 'Contact Sales'. Below the summary, a table lists the services included in the estimate:

Service Name	Status	Upfront cost	Monthly cost	Description	Region	Config Summary
AWS Lambda	-	0.00 USD	0.00 USD	-	US East (N. Virginia)	Architecture (x86), Architec...
Amazon API Gateway	-	0.00 USD	0.01 USD	-	US East (N. Virginia)	HTTP API requests units (e...
Amazon DynamoDB	-	0.00 USD	0.00 USD	-	US East (N. Virginia)	Table class (Standard), Ave...
Amazon Simple Storage Se...	-	0.00 USD	0.07 USD	-	US East (N. Virginia)	S3 Standard storage (1 GB ...
Amazon CloudFront	-	0.00 USD	0.00 USD	-	US East (N. Virginia)	Free Plan (1)
Amazon EC2	-	0.00 USD	7.59 USD	-	US East (N. Virginia)	Tenancy (Shared Instances)...

9. Buenas prácticas aplicadas

- Seguridad: Apliqué el principio de menor privilegio en todos los roles IAM: Quotick-LambdaRole solo puede leer y escribir en las tablas quotick-*, y Quotick-EC2Role solo puede subir archivos a la carpeta pdfs/ del bucket. El acceso a la EC2 no requiere llave SSH ni puerto 22 abierto; uso AWS Systems Manager Session Manager, que autentica con IAM.
- Infraestructura como código: Todo el backend está descrito en serverless.yml: funciones, rutas, variables de entorno y timeouts. La infraestructura de red (VPC, subred, IGW, SG, EC2, IAM) se crea con scripts de AWS CLI versionados en infrastructure/. Esto permite recrear todo el entorno desde cero en minutos.
- Sin valores hardcodeados: Los nombres de las tablas DynamoDB se pasan a Lambda como variables de entorno definidas en serverless.yml. La TRM, las horas por día, los días por mes y el porcentaje de utilización son registros

editables en DynamoDB, no constantes en el código. Cualquier cambio de parámetro se hace desde la propia app.

- Convención de nombres: Todos los recursos AWS llevan el prefijo quotick-* (tablas, bucket, roles, VPC, EC2). Esto facilita la identificación en la consola, simplifica las políticas IAM con wildcards (quotick-*) y hace más fácil limpiar o auditar los recursos.
- Separación de capas: Frontend, backend e infraestructura de red son tres capas independientes con ciclos de despliegue separados. Actualizar el frontend no afecta el backend y viceversa. Esto reduce el riesgo de romper algo al hacer cambios y facilita el trabajo en paralelo.

10. Conclusiones y aprendizajes

Construir Quotick me obligó a tomar decisiones de arquitectura reales, no solo académicas. La más interesante fue descubrir que serverless y EC2 no son opciones excluyentes: el 99% del sistema corre en servicios administrados que escalan solos, y la única excepción tiene una justificación técnica concreta (el límite de tamaño de paquete de Lambda). Eso es lo que en la práctica se llama arquitectura híbrida, y es un patrón común en proyectos reales.

Entendí por qué DynamoDB on-demand y Lambda funcionan tan bien juntos: ambos escalan automáticamente, ambos tienen modelo de pago por uso, y ninguno requiere aprovisionar capacidad de antemano. Para un sistema con tráfico bajo o variable, esa combinación es imbatible en costo.

También aprendí a gestionar IAM con seriedad. Al principio es tentador dar permisos amplios para evitar errores, pero definir exactamente qué puede hacer cada rol hace el sistema más seguro y fácil de auditar.

Por último, usar Serverless Framework como IaC me enseñó el valor de describir la infraestructura en código: desplegar 25 funciones Lambda con sus rutas y variables de entorno es un solo comando. Eso es algo que no se aprecia hasta que tienes que hacerlo a mano una vez.